

Application Deployment Guide

Document Purpose: This guide provides step-by-step instructions for deploying the application using Docker and Docker Compose on an AWS t3.large instance (or equivalent infrastructure).

Prerequisites

Before beginning the deployment, ensure the following are in place:

- ✓ AWS t3.large instance (or equivalent) provisioned and running
 - ✓ Docker Engine (version 20.10+) installed
 - ✓ Docker Compose (version 2.0+) installed
 - ✓ Domain name configured with DNS A record pointing to your server's public IP
 - ✓ Ports 80 (HTTP) and 443 (HTTPS) open in security group/firewall
 - ✓ SSH access to the server
 - ✓ Root or sudo privileges
-

Step 1: Connect to Your Server

Connect to your server via SSH:

```
bash  
ssh username@your-server-ip
```

Step 2: Prepare the Deployment Directory

Create and navigate to the deployment directory:

```
bash  
mkdir -p /opt/application  
cd /opt/application
```

Step 3: Transfer Deployment Files

Transfer the following files to your server (use `scp`, `rsync`, or clone from repository):

- `docker-compose.yml` - Docker Compose configuration file
- `request_ssl.sh` - SSL certificate request script
- `.env.example` - Environment variables template (if provided)
- Any additional configuration files

Example using scp:

```
bash

# From your local machine
scp docker-compose.yml username@your-server-ip:/opt/application/
scp request_ssl.sh username@your-server-ip:/opt/application/
```

Or clone from repository:

```
bash

git clone <repository-url> /opt/application
cd /opt/application
```

Step 4: Configure Environment Variables

4.1 Create Environment File

Copy the `.env.example` file to create your environment file:

```
bash

cp .env.example .env
```

4.2 Set the `SERVER_ADDRESS` Variable

Edit the environment file or export the variable:

Option A: Using `.env` file

```
bash

nano .env
```

Add or modify:

```
SERVER_ADDRESS=your-domain.com
```

Option B: Export as environment variable

```
bash
export SERVER_ADDRESS=your-domain.com
```

Replace `your-domain.com` with your actual domain name (e.g., `app.example.com`).

4.3 Configure Additional Variables

Set any other required environment variables in the `.env` file:

- Database credentials
 - API keys
 - Application-specific configurations
-

Step 5: Make Scripts Executable

Ensure the SSL request script has execute permissions:

```
bash
chmod +x request_ssl.sh
```

Step 6: Request SSL Certificate

Run the SSL certificate request script to obtain a free Let's Encrypt certificate:

```
bash
./request_ssl.sh
```

What this script does:

- Validates your domain configuration
- Requests an SSL/TLS certificate from Let's Encrypt
- Stores certificates in the appropriate directory for Docker volume mounting
- Configures automatic certificate renewal

Expected output:

- Confirmation message indicating successful certificate issuance

- Certificate files created (typically in `./certbot/conf/live/your-domain.com/`)

Troubleshooting:

- Ensure your domain's DNS A record points to your server's public IP
 - Verify ports 80 and 443 are accessible
 - Check that no other service is using port 80 during certificate request
-

Step 7: Review Docker Compose Configuration

Before deploying, review your `docker-compose.yml` file:

```
bash
cat docker-compose.yml
```

Verify:

- Service definitions are correct
 - Volume mounts for SSL certificates are properly configured
 - Environment variables are referenced correctly
 - Port mappings are appropriate (typically 80:80 and 443:443)
 - Network configuration is suitable for your setup
-

Step 8: Deploy the Application

Start all services using Docker Compose:

```
bash
docker-compose up -d
```

What this command does:

- `-d` flag runs containers in detached mode (background)
- Pulls required Docker images from registries
- Creates and starts all services defined in `docker-compose.yml`
- Mounts SSL certificates into appropriate containers
- Configures Nginx to use HTTPS with your certificates

- Sets up networking between services
-

Step 9: Verify Deployment

9.1 Check Container Status

Verify all containers are running:

```
bash
docker-compose ps
```

Expected output: All services should show status as "Up"

9.2 Check Container Logs

Monitor logs for any errors:

```
bash

# View all service logs
docker-compose logs

# View logs for a specific service
docker-compose logs <service-name>

# Follow logs in real-time
docker-compose logs -f
```

9.3 Test Application Access

Test your application access:

HTTP Redirect Test:

```
bash
curl -I http://your-domain.com
```

Should redirect to HTTPS (301 or 302 response)

HTTPS Access Test:

```
bash
curl -I https://your-domain.com
```

Should return 200 OK

Browser Test:

- Open `https://your-domain.com` in your web browser
 - Verify the SSL certificate is valid (padlock icon)
 - Confirm the application loads correctly
-

Step 10: Configure Automatic Startup

Ensure containers restart automatically after system reboot:

```
bash

docker-compose up -d --restart unless-stopped
```

Or verify the restart policy in your `docker-compose.yml`:

```
yaml

services:
  service-name:
    restart: unless-stopped
```

Post-Deployment Tasks

SSL Certificate Renewal

Let's Encrypt certificates expire after 90 days. Set up automatic renewal:

Option A: Using Certbot container (if included in `docker-compose.yml`) The certbot container should handle automatic renewal.

Option B: Manual cron job

```
bash

# Edit crontab
crontab -e

# Add renewal job (runs daily at 2 AM)
0 2 * * * cd /opt/application && ./request_ssl.sh renew && docker-compose restart nginx
```

Backup Configuration

Create regular backups of:

- Environment files (`.env`)
- Docker Compose configuration (`docker-compose.yml`)
- SSL certificates (`./certbot/conf/`)
- Persistent data volumes

Monitoring Setup

Implement monitoring for:

- Container health status
- Resource utilization (CPU, memory, disk)
- Application logs
- SSL certificate expiration

Useful Commands

View running containers

```
bash
docker-compose ps
```

Stop all services

```
bash
docker-compose down
```

Restart specific service

```
bash
docker-compose restart <service-name>
```

View real-time logs

```
bash
```

```
docker-compose logs -f <service-name>
```

Update and restart services

```
bash  
  
docker-compose pull  
docker-compose up -d
```

Remove all containers and volumes (CAUTION: deletes data)

```
bash  
  
docker-compose down -v
```

Troubleshooting

Issue: Containers fail to start

Solution:

```
bash  
  
# Check logs for errors  
docker-compose logs  
  
# Verify environment variables  
cat .env  
  
# Check port conflicts  
sudo netstat -tulpn | grep -E ':(80|443)'
```

Issue: SSL certificate request fails

Solution:

- Verify DNS propagation: `nslookup your-domain.com`
- Check firewall rules: `sudo ufw status`
- Ensure port 80 is accessible for Let's Encrypt validation
- Review request_ssl.sh logs

Issue: Application not accessible via HTTPS

Solution:


```
bash

# Verify Nginx configuration
docker-compose exec nginx nginx -t

# Check certificate files
ls -la ./certbot/conf/live/your-domain.com/

# Restart Nginx service
docker-compose restart nginx
```

Issue: Permission denied errors

Solution:

```
bash

# Fix ownership of deployment directory
sudo chown -R $USER:$USER /opt/application

# Ensure Docker socket permissions
sudo usermod -aG docker $USER
newgrp docker
```

Security Best Practices

1. **Secure environment variables:** Never commit `.env` files to version control
2. **Regular updates:** Keep Docker images and system packages updated
3. **Firewall configuration:** Only expose necessary ports (80, 443, 22)
4. **SSH hardening:** Use SSH keys, disable password authentication, change default port
5. **Log monitoring:** Regularly review application and system logs
6. **Backup strategy:** Implement automated backups of critical data
7. **Access control:** Limit SSH access to specific IP addresses if possible

Support and Maintenance

Getting Help

If you encounter issues during deployment:

1. Review container logs: `docker-compose logs`
2. Check system resources: `htop` or `docker stats`
3. Verify network connectivity
4. Consult the troubleshooting section above

Regular Maintenance

Perform these tasks regularly:

- **Weekly:** Review application logs for errors
 - **Monthly:** Update Docker images and restart services
 - **Quarterly:** Verify backups and disaster recovery procedures
 - **As needed:** Scale resources based on usage patterns
-

Document Version: 1.0

Last Updated: October 15, 2025

For technical support or questions about this deployment, please contact your deployment team.